ROTEIRO PRÁTICO 1 • FASE 1 DO PROJETO

Versionamento de Código e Integração Contínua com GitLab CI

Do código no seu computador até um pipeline de build, lint e testes rodando a cada push

Competência: Elaborar um pipeline de CI/CD para cloud computing

Professora: Ma. Sheila Maria Mendes Paiva

Aplicação de apoio: API Catálogo de Filmes (Python + Flask)

Como usar este roteiro

Este roteiro é um passo a passo guiado. Cada etapa explica **o que** você vai fazer, **por que** está fazendo e **como verificar** que deu certo. Você vai usar uma API pronta, a **API Catálogo de Filmes**, para praticar exatamente o que é pedido na **Fase 1 do projeto**. Depois, basta repetir os mesmos passos com a API que o seu grupo vai desenvolver.

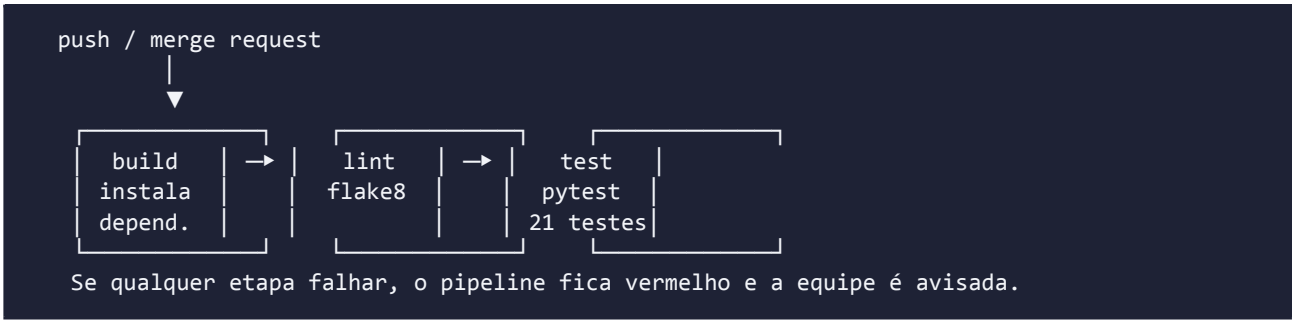
> ⚠️ **ATENÇÃO**
> A API de filmes é apenas um **exemplo para praticar**. O projeto do grupo deve ter um tema próprio, aprovado pela professora. Copiar a API de exemplo ou o pipeline sem adaptação e compreensão é considerado plágio (nota zero na fase).

Ao longo do texto você vai encontrar caixas coloridas:

Caixa	Para que serve
📘 CONCEITO	Relembra a teoria vista em aula, ligada àquele passo.
💡 DICA	Atalhos e boas práticas que poupam tempo.
⚠️ ATENÇÃO	Erros comuns que fazem o pipeline (ou a nota) falhar.
✅ CHECKPOINT	Como confirmar que a etapa funcionou antes de seguir.
🎯 NO SEU PROJETO	O que você precisa replicar no projeto do grupo.

O que você vai construir

Ao final deste roteiro você terá um repositório no GitLab organizado com branches e Merge Requests, e um pipeline de Integração Contínua com três estágios:



Pré-requisitos

Ferramenta	Como verificar	Observação
Git	`git --version`	Qualquer versão 2.x
Python 3.10 ou superior	`python3 --version` (Windows: `python --version`)	Recomendado 3.12, a mesma usada no pipeline
Conta no GitLab.com	Acesse gitlab.com e faça login	Cada integrante do grupo precisa da sua
Editor de código	VS Code ou similar	Com suporte a YAML ajuda bastante

Etapa 1: Conhecendo a API de exemplo

Descompacte a pasta `api-filmes` que acompanha este roteiro. A estrutura é esta:

```
api-filmes/
├── app/
│   ├── __init__.py      # cria a aplicação Flask (application factory)
│   ├── models.py        # modelo Filme (tabela "filmes")
│   └── routes.py        # as 6 rotas da API
├── tests/
│   ├── conftest.py      # fixtures: app de teste com banco em memória
│   ├── test_health.py   # 3 testes do healthcheck
│   └── test_filmes.py   # 18 testes das rotas de filmes
├── wsgi.py              # ponto de entrada da aplicação
├── requirements.txt     # dependências com versões fixas
├── .flake8              # configuração do lint
├── .gitignore           # o que NÃO deve ir para o repositório
└── README.md
```

1.1 As rotas

A API representa um único caso de uso coerente (um catálogo de filmes), como exige o projeto. São 6 rotas:

Método	Rota	O que faz	Respostas possíveis
GET	/health	Healthcheck: confirma que a API está no ar	200
GET	/filmes	Lista os filmes (aceita `?genero=Drama`)	200
GET	/filmes/<id>	Busca um filme pelo id	200, 404
POST	/filmes	Cadastra um filme	201, 400
PUT	/filmes/<id>	Atualiza um ou mais campos	200, 400, 404
DELETE	/filmes/<id>	Remove um filme	204, 404

1.2 Como as rotas são escritas

Abra `app/routes.py`. Cada rota é uma função Python decorada com o método HTTP e o caminho. Veja a rota de busca por id:

```
app/routes.py (trecho)
@bp.get("/filmes/<int:filme_id>")
def buscar_filme(filme_id):
    filme = db.session.get(Filme, filme_id)
    if filme is None:
        return jsonify({"erro": "Filme não encontrado"}), 404
    return jsonify(filme.to_dict()), 200
```

Repare que a rota trata o caso de erro (filme inexistente → **404**). Na rota de criação, a função `validar_filme` verifica os dados e devolve **400** quando algo está faltando ou é inválido. São exatamente esses comportamentos que os testes vão verificar.

1.3 Como os testes são escritos

Os testes usam o **pytest** e o cliente de testes do Flask, que faz requisições à API sem precisar subir um servidor. O arquivo `tests/conftest.py` cria uma aplicação nova, com um banco SQLite **em memória**, para cada teste. Assim um teste nunca interfere no outro.

tests/conftest.py

```
"""Fixtures compartilhadas pelos testes."""
import pytest

from app import create_app
from app.models import db


@pytest.fixture
def app():
    # Cada teste usa um banco SQLite em memória, limpo e isolado.
    app = create_app({
        "TESTING": True,
        "SQLALCHEMY_DATABASE_URI": "sqlite:///:memory:",
    })
    yield app
    with app.app_context():
        db.drop_all()


@pytest.fixture
def client(app):
    return app.test_client()


@pytest.fixture
def filme_criado(client):
    """Cria um filme e devolve o JSON retornado pela API."""
    resposta = client.post("/filmes", json={
        "titulo": "Central do Brasil",
        "ano": 1998,
        "genero": "Drama",
        "nota": 9.1,
    })
    return resposta.get_json()
```

Cada rota tem pelo menos 3 testes cobrindo casos diferentes: sucesso, erro de validação (400) e recurso inexistente (404). Exemplos:

tests/test_filmes.py (trecho)

```
def test_criar_filme_valido(client):
    resposta = client.post("/filmes", json=FILME_VALIDO)
    assert resposta.status_code == 201
    dados = resposta.get_json()
    assert dados["titulo"] == "Cidade de Deus"


def test_criar_filme_sem_campo_obrigatorio_retorna_400(client):
    resposta = client.post("/filmes", json={"titulo": "Sem ano"})
    assert resposta.status_code == 400


def test_buscar_filme_inexistente_retorna_404(client):
    resposta = client.get("/filmes/999")
    assert resposta.status_code == 404
```

📘 **CONCEITO**

Um bom teste **falha quando algo quebra**. Um teste como `assert True` sempre passa e não protege nada. Os `assert` acima comparam o resultado real com o esperado; se a API mudar de comportamento, o teste fica vermelho.

🎯 **NO SEU PROJETO**

- Ao menos **5 rotas** (1 por aluno) de um mesmo domínio.
- Ao menos **3 testes por rota** (mínimo 15), cobrindo casos diferentes.
- POST/PUT: incluir teste de **dados inválidos** (400).
- GET por id: incluir teste de **recurso inexistente** (404).
- Um teste de **healthcheck**.

Etapa 2: Rodando a API e os testes no seu computador

Antes de automatizar qualquer coisa, é preciso saber executar tudo manualmente. O pipeline vai apenas repetir, em um ambiente limpo, os mesmos comandos que você roda aqui.

2.1 Criar o ambiente virtual e instalar as dependências

```
$ cd api-filmes
$ python3 -m venv venv
$ source venv/bin/activate        # Linux / macOS
# venv\Scripts\activate          # Windows (PowerShell ou CMD)
$ pip install -r requirements.txt
```

> 📘 **CONCEITO**
>
> O **ambiente virtual** (venv) isola as bibliotecas deste projeto das do restante do sistema. O arquivo `requirements.txt` fixa as versões (por exemplo `Flask==3.1.0`), garantindo que todos do grupo e o pipeline usem exatamente as mesmas dependências. Isso combate o famoso "na minha máquina funciona" (Aula 1).

2.2 Subir a API

```
$ python wsgi.py
 * Running on http://127.0.0.1:5000
```

Em outro terminal, faça algumas requisições (ou use Postman/Insomnia):

```
$ curl http://localhost:5000/health
{"status": "ok"}

$ curl -X POST http://localhost:5000/filmes \
       -H "Content-Type: application/json" \
       -d '{"titulo": "Bacurau", "ano": 2019, "genero": "Suspense", "nota": 8.5}'
{"ano": 2019, "genero": "Suspense", "id": 1, "nota": 8.5, "titulo": "Bacurau"}

$ curl http://localhost:5000/filmes/1
$ curl http://localhost:5000/filmes/999      # deve retornar 404
```

> 💡 **DICA**
>
> No Windows (PowerShell) o `curl` pode se comportar diferente. Use o **Postman/Insomnia** ou o comando `Invoke-RestMethod`.

2.3 Rodar os testes e o lint

```
$ pytest -v
...
tests/test_filmes.py::test_criar_filme_valido PASSED
tests/test_filmes.py::test_buscar_filme_inexistente_retorna_404 PASSED
...
```

```
============ 21 passed in 0.25s ============

$ flake8 .
$                                  # nenhuma saída = nenhum problema
```

> 📘 **CONCEITO**
> **Lint** é a análise estática do código: o `flake8` lê o código **sem executá-lo** e aponta problemas de estilo (PEP 8), variáveis não usadas, imports sobrando, linhas longas etc. O arquivo `.flake8` define as regras, por exemplo `max-line-length = 100`.

2.4 Faça um teste quebrar de propósito

Abra `app/routes.py` e, na rota `buscar_filme`, troque `404` por `200`. Rode `pytest` de novo:

```
$ pytest
FAILED tests/test_filmes.py::test_buscar_filme_inexistente_retorna_404
    assert 200 == 404
============ 1 failed, 20 passed ============
```

É exatamente isso que o pipeline vai fazer por você a cada push. **Desfaça a alteração** antes de continuar.

> ✅ **CHECKPOINT**
> - A API responde em `http://localhost:5000/health`.
> - `pytest` mostra **21 passed**.
> - `flake8 .` não mostra nenhuma mensagem.

Etapa 3: Criando o repositório no GitLab

3.1 Criar o projeto

1. No GitLab, clique em **New project** → **Create blank project**.
2. Dê um nome (ex.: `api-filmes-<seu-nome>`), mantenha a visibilidade **Private**.
3. **Desmarque** a opção *Initialize repository with a README* (vamos enviar um projeto que já existe).
4. Clique em **Create project** e copie a URL HTTPS do repositório (botão **Code**).

3.2 Primeiro commit e push

Dentro da pasta `api-filmes`:

```
$ git init --initial-branch=main
$ git status                      # arquivos aparecem como "untracked"
$ git add .
$ git status                      # agora estão na staging area
$ git commit -m "feat: estrutura inicial da API Catálogo de Filmes"
$ git remote add origin https://gitlab.com/<usuario>/api-filmes-<seu-nome>.git
$ git push -u origin main
```

> 📘 **CONCEITO**
> Lembre das **três áreas do Git** (Aula 3): o `git add` leva as mudanças do **diretório de trabalho** para a **staging area**; o `git commit` grava um snapshot no **repositório local**; o `git push` envia os commits para o **repositório remoto** (GitLab).

> ⚠️ **ATENÇÃO**
> Confira que a pasta `venv/` **não** foi enviada. Ela está listada no `.gitignore`. Ambientes virtuais, `__pycache__`, bancos `.db` e arquivos `.env` nunca devem ir para o repositório.
>
> Se o GitLab pedir senha no push, use um **Personal Access Token** (Preferences → Access tokens, escopo `write_repository`) no lugar da senha.

3.3 Adicionar os colegas e proteger a branch main

1. **Manage** → **Members** → **Invite members**: adicione os integrantes do grupo com papel **Developer** (ou Maintainer) e a professora com papel **Reporter** ou superior.
2. **Settings** → **Repository** → **Protected branches**: na branch `main`, deixe *Allowed to push and merge* = **No one** e *Allowed to merge* = **Maintainers**. Assim ninguém faz push direto na `main`: tudo entra via Merge Request.
3. **Settings** → **Merge requests** → **Merge checks**: marque **Pipelines must succeed**. Um MR com pipeline vermelho não pode ser integrado.

3.4 Estratégia de branches

Vamos usar um fluxo simples, suficiente para o projeto:






```
$ git checkout -b develop
$ git push -u origin develop
```

> 💡 **DICA**
>
> Padrão de mensagens de commit: use o modo imperativo e um prefixo que diga o tipo da mudança.
>
> - `feat: adiciona rota de busca por gênero`
> - `fix: corrige validação da nota acima de 10`
> - `test: adiciona testes da rota DELETE`
> - `ci: adiciona estágio de lint ao pipeline`
> - `docs: atualiza instruções do README`
>
> Evite mensagens como "ajustes", "funcionando agora" ou "fix".

> ✅ **CHECKPOINT**
>
> - O código aparece em **Code** → **Repository**, nas branches `main` e `develop`.
> - A `main` aparece com o ícone de cadeado (protegida).

Etapa 4: Construindo o pipeline de CI passo a passo

O pipeline é definido em um único arquivo, `.gitlab-ci.yml`, **na raiz do repositório**. O GitLab lê esse arquivo a cada push e distribui os jobs para um **Runner**, que executa os comandos em um container isolado. Vamos construí-lo em incrementos, testando cada um.

> ⚠️ **ATENÇÃO**
>
> No GitLab.com, para usar os runners compartilhados (gratuitos) pode ser exigida a **verificação da conta** (telefone ou cartão, sem cobrança). Se o pipeline ficar travado em *pending* ou aparecer um aviso de validação, siga as instruções da tela ou fale com a professora.
>
> Confira também em **Settings** → **CI/CD** → **Runners** se a opção **Enable instance runners** está ativa.

> 💡 **DICA**
>
> Use o editor do próprio GitLab em **Build** → **Pipeline editor**: ele valida a sintaxe YAML antes do commit e mostra a visualização do pipeline. Lembre: em YAML a indentação é feita com **espaços, nunca tabs**.

4.1 Incremento 1: o menor pipeline possível

Crie a branch da tarefa e o arquivo `.gitlab-ci.yml` com apenas um job:

```
$ git checkout develop
$ git checkout -b feature/pipeline-ci
```

```
.gitlab-ci.yml (versão 1)
image: python:3.12-slim

ola:
  script:
    - python --version
    - echo "Meu primeiro pipeline!"
```

```
$ git add .gitlab-ci.yml
$ git commit -m "ci: cria primeiro pipeline"
$ git push -u origin feature/pipeline-ci
```

Abra **Build** → **Pipelines**: deve existir um pipeline com o job `ola`. Clique no job para ver o **log**, que mostra cada comando executado, como se fosse o seu terminal.

> 📘 **CONCEITO**
>
> - `image`: imagem Docker que serve de ambiente para os jobs (aqui, um Linux com Python 3.12).
> - **Job** (`ola`): unidade de trabalho; precisa obrigatoriamente de `script`.
> - `script`: lista de comandos. Se **qualquer** comando terminar com erro, o job falha.
> - Sem `stages` declarados, o job vai para o stage padrão `test`.

4.2 Incremento 2: stages e o job de build

Substitua o conteúdo do arquivo:

```
.gitlab-ci.yml (versão 2)
image: python:3.12-slim

stages:
  - build

build:
  stage: build
  script:
    - python -m venv venv
    - source venv/bin/activate
    - pip install -r requirements.txt
    # "Compila" todos os arquivos .py: falha se houver erro de sintaxe
    - python -m compileall -q app
    # Garante que a aplicação consegue ser criada (imports ok)
    - python -c "from app import create_app; create_app()"
```

Python é uma linguagem interpretada, então o "build" aqui significa **preparar o ambiente** (instalar dependências) e **verificar** que o código é carregável. Em Java, por exemplo, seria o `mvn package` que gera o `.jar` (Aula 5).

4.3 Incremento 3: lint e test, usando artifacts

Cada job roda em um container **novo e vazio**. Para não reinstalar tudo em cada job, o `build` vai entregar a pasta `venv/` como **artifact** para os jobs seguintes:

```
.gitlab-ci.yml (versão 3)
image: python:3.12-slim

stages:
  - build
  - lint
  - test

build:
  stage: build
  script:
    - python -m venv venv
    - source venv/bin/activate
    - pip install -r requirements.txt
    - python -m compileall -q app
    - python -c "from app import create_app; create_app()"
  artifacts:
    paths:
      - venv/
    expire_in: 1 hour

lint:
  stage: lint
  script:
    - source venv/bin/activate
    - flake8 .

test:
  stage: test
  script:
    - source venv/bin/activate
    - pytest -v
```

📘 **CONCEITO**

- `stages` define a **ordem** das etapas. Um stage só começa se o anterior terminou com sucesso.
- Jobs do **mesmo** stage rodam em paralelo.
- `artifacts` são arquivos **produzidos** por um job e repassados aos jobs dos stages seguintes (e que podem ser baixados na interface). `expire_in` evita acumular arquivos para sempre.

✅ **CHECKPOINT**

Faça commit e push. Em **Build** → **Pipelines** você deve ver 3 colunas (build → lint → test), todas verdes.

4.4 Incremento 4: cache, relatório de testes e regras de execução

Agora a versão final da Fase 1:

```
.gitlab-ci.yml (versão final da Fase 1)
# ============================================================
# Pipeline da FASE 1: build -> lint -> test
# ============================================================
image: python:3.12-slim

stages:
  - build
  - lint
  - test

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"

# O cache guarda os pacotes baixados pelo pip entre execuções,
# deixando o pipeline mais rápido.
cache:
  key:
    files:
      - requirements.txt
  paths:
    - .cache/pip

# Roda a cada push e a cada Merge Request (evitando pipeline duplicado)
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH

build:
  stage: build
  script:
    - python -m venv venv
    - source venv/bin/activate
    - pip install -r requirements.txt
    - python -m compileall -q app
    - python -c "from app import create_app; create_app()"
  artifacts:
    paths:
```

```
      - venv/
    expire_in: 1 hour

lint:
  stage: lint
  script:
    - source venv/bin/activate
    - flake8 .

test:
  stage: test
  script:
    - source venv/bin/activate
    - pytest -v --junitxml=report.xml
  artifacts:
    when: always
    reports:
      junit: report.xml
```

O que mudou:

Palavra-chave	Para que serve aqui
`variables` + `cache`	O `pip` guarda os pacotes baixados em `.cache/pip`. O `cache` reaproveita essa pasta entre pipelines, deixando as próximas execuções mais rápidas. A chave (`key: files: requirements.txt`) faz o cache ser renovado só quando as dependências mudam.
`workflow: rules`	Define **quando** o pipeline roda: a cada push em qualquer branch e a cada Merge Request. A regra do meio evita pipelines duplicados quando a branch já tem um MR aberto.
`pytest --junitxml`	Gera um relatório que o GitLab exibe na aba **Tests** do pipeline e dentro do Merge Request.
`when: always`	Guarda o relatório mesmo quando o job falha (justamente quando ele é mais útil).

📘 **CONCEITO**

Cache × Artifacts (cai na prova!)

- **Cache**: otimização de velocidade; guarda dependências baixadas entre execuções **diferentes** do pipeline. Não há garantia de que estará disponível.
- **Artifacts**: resultado de um job, repassado de forma **garantida** aos stages seguintes do **mesmo** pipeline (ex.: o `venv/`, um `.jar`, um relatório).

Etapa 5: O fluxo completo com Merge Request

5.1 Abrir o Merge Request do pipeline

1. Em **Code** → **Merge requests** → **New merge request**, escolha origem `feature/pipeline-ci` e destino `develop`.
2. Descreva o que foi feito, atribua um colega como **Reviewer** e crie o MR.
3. Observe que o MR mostra o status do pipeline e, após rodar, a aba de testes.
4. O revisor comenta, aprova e clica em **Merge**.

5.2 Praticando: adicione uma nova rota com testes

Agora simule o trabalho de um integrante do grupo. Crie a rota `GET /filmes/melhores`, que retorna os filmes com nota maior ou igual a 8, ordenados da maior para a menor nota.

```
$ git checkout develop
$ git pull
$ git checkout -b feature/rota-melhores
```

```
app/routes.py (adicione antes da rota buscar_filme)
@bp.get("/filmes/melhores")
def melhores_filmes():
    """Filmes com nota >= 8, do melhor para o pior."""
    filmes = (
        Filme.query.filter(Filme.nota >= 8)
        .order_by(Filme.nota.desc())
        .all()
    )
    return jsonify([f.to_dict() for f in filmes]), 200
```

```
tests/test_filmes.py (adicione ao final)
def test_melhores_lista_vazia(client):
    resposta = client.get("/filmes/melhores")
    assert resposta.status_code == 200
    assert resposta.get_json() == []


def test_melhores_ignora_notas_baixas(client):
    client.post("/filmes", json={**FILME_VALIDO, "nota": 5})
    resposta = client.get("/filmes/melhores")
    assert resposta.get_json() == []


def test_melhores_ordena_por_nota(client):
    client.post("/filmes", json={**FILME_VALIDO, "titulo": "A", "nota": 8.5})
    client.post("/filmes", json={**FILME_VALIDO, "titulo": "B", "nota": 9.7})
    titulos = [f["titulo"] for f in client.get("/filmes/melhores").get_json()]
    assert titulos == ["B", "A"]
```

```
$ pytest && flake8 .                # sempre valide localmente antes do push
$ git add .
$ git commit -m "feat: adiciona rota de melhores filmes"
$ git push -u origin feature/rota-melhores
```

Abra o MR para `develop`, aguarde o pipeline ficar verde e faça o merge. Quando o grupo tiver uma versão estável, abra um MR de `develop` para `main`.

5.3 Simulando falhas (e aprendendo a ler o log)

Faça os dois experimentos abaixo em uma branch de teste, um de cada vez:

Experimento	O que fazer	Resultado esperado
Teste quebrado	Na validação da nota, troque `nota > 10` por `nota > 100`.	Job **test** falha. O log mostra qual `assert` falhou. Os stages seguintes não rodam.
Erro de lint	Adicione `import os` sem usar em `routes.py`.	Job **lint** falha com `F401 'os' imported but unused`. O **test** nem chega a rodar.

Observe que o Merge Request fica **bloqueado** para merge enquanto o pipeline estiver vermelho (por causa do *Pipelines must succeed*). Corrija, faça um novo push e veja o pipeline voltar a ficar verde.

> 📘 **CONCEITO**
> Isso é o "**falhar rápido**" da Integração Contínua (Aula 4): o erro é detectado minutos depois do commit, por quem acabou de escrever o código, e não semanas depois, quando ninguém lembra mais o que mudou.

> ✅ **CHECKPOINT**
> - Existe pelo menos um MR integrado em `develop` e um de `develop` para `main`.
> - O último pipeline da `main` está verde (**Build** → **Pipelines**).
> - A aba **Tests** do pipeline mostra os testes executados.

Etapa 6: Replicando no projeto do grupo

Use a tabela abaixo como checklist antes da entrega. Ela segue os critérios de avaliação da Fase 1.

Critério (pontos)	O que precisa existir no repositório	Onde praticou
Organização do repositório (0,5)	README com descrição, tecnologias e como rodar; branch de desenvolvimento; integração via MR; commits claros; API com ≥ 5 rotas e testes conforme os requisitos.	Etapas 1, 3 e 5
Pipeline de CI funcional (0,5)	`.gitlab-ci.yml` versionado na raiz; pipeline roda sem erros a cada push e MR.	Etapa 4
Estágio de build (0,5)	Job que instala as dependências e verifica/empacota a aplicação.	4.2
Estágio de testes (0,5)	Job que executa o pytest e **falha** se um teste quebrar.	4.3 e 5.3
Estágio de lint (0,5)	Job que executa o flake8.	4.3 e 5.3
Stages/artifacts/cache (0,5)	Stages em ordem lógica; uso adequado de artifacts e/ou cache.	4.3 e 4.4

🎯 **NO SEU PROJETO**

- Cada integrante deve ter **commits relevantes** (por exemplo, a sua rota e os seus testes, em uma branch própria, integrada por MR).
- Todos devem saber explicar **cada linha** do `.gitlab-ci.yml`: haverá perguntas individuais na apresentação.
- Se o grupo usar **FastAPI** em vez de Flask, troque o cliente de testes por `fastapi.testclient.TestClient`; o pipeline praticamente não muda.

Problemas comuns

Sintoma	Causa provável e solução
`This GitLab CI configuration is invalid`	Erro de YAML: tab no lugar de espaços, indentação errada ou falta de `:`. Valide no **Pipeline editor**.
Pipeline fica em *pending* para sempre	Sem runner disponível: verifique a validação da conta e **Settings** → **CI/CD** → **Runners**.
`source: not found`	A imagem usa `sh` sem `source`. Use uma imagem `python:*` (que tem bash) ou troque por `. venv/bin/activate`.
`flake8: command not found` no job lint	O artifact `venv/` não chegou: confira `artifacts: paths` no build e se o build terminou com sucesso.
`ModuleNotFoundError: No module named 'app'`	O pytest não está sendo executado da raiz do projeto, ou falta `tests/__init__.py`.
Push rejeitado na `main`	A branch está protegida (correto!). Faça push na sua feature branch e abra um MR.
Pipelines duplicados no MR	Falta a regra de `workflow` da versão final (Etapa 4.4).

Desafios extras (opcionais)

- Adicione cobertura de testes com `pytest-cov` e exiba o percentual no MR (`coverage: '/TOTAL.*\s+(\d+%)$/'`).
- Crie um job que só roda em Merge Requests usando `rules: - if: $CI_PIPELINE_SOURCE == "merge_request_event"`.
- Adicione um badge de status do pipeline ao README (**Settings** → **CI/CD** → **General pipelines** → **Pipeline status**).